

PostgreSQL CDC with Debezium

Debezium captures row-level changes in PostgreSQL by reading the write-ahead log (WAL) through PostgreSQL's logical decoding feature, not the table directly. A logical decoding *output plug-in* on the server side turns WAL entries into a consumable stream; Debezium's Java connector reads that stream over the standard replication protocol and emits Kafka events. Unlike Mongo (where the driver already gives you **before/after** as embedded JSON strings), Postgres logical decoding gives you actual structured row data per plug-in.

1. PostgreSQL side — admin setup runbook

1.1 Choose a plug-in

- **pgoutput** — built into Postgres 10+, no install, Postgres-maintained. Use this unless you have a specific reason not to; it's the only option on every managed Postgres (RDS, Cloud SQL, Azure, CrunchyBridge) anyway.
- **decoderbufs** — Debezium-maintained, Protobuf-based, requires installing a separate extension. Legacy at this point.

This doc uses **pgoutput** from here on. Nothing to run for this step — it's a decision that shows up later as **plugin.name**.

1.2 Enable logical decoding

In **postgresql.conf** (requires a full server restart, not just a reload):

```
wal_level = logical
max_wal_senders = 4
max_replication_slots = 4
```

- **wal_level = logical** — WAL entries include enough info to reconstruct row changes, not just physical replication. Mandatory; nothing works without it.
- **max_wal_senders** — number of concurrent processes allowed to stream WAL out. One per connector/slot in use, plus headroom.
- **max_replication_slots** — cap on how many slots can exist at once. Each Debezium connector owns exactly one slot.

Optional but worth knowing: **wal_keep_size** caps how much WAL a slot can force the server to retain before things get ugly (see the disk-bloat note later), and **wal_writer_delay** (default 200ms) adds latency if **synchronous_commit** isn't **on** — drop it to something like 10ms if you care about event lag.

Verify:

```
SHOW wal_level; SHOW max_wal_senders; SHOW max_replication_slots;
```

On managed Postgres you usually can't touch `postgresql.conf` directly — there's an instance-level flag instead, and it still needs a restart to take effect:

- **RDS:** set `rds.logical_replication = 1` on the parameter group, restart, then `SHOW wal_level` as the master user. Grant `rds_replication` to any non-master account that needs to run the connector or take the initial snapshot.
- **Cloud SQL:** set the `cloudsql.logical_decoding` flag to `on` — this auto-flips `wal_level` to `logical` for you.
- **Azure Database for PostgreSQL:** `az postgres server configuration set --name azure.replication_support --value logical`, then restart.

All three (plus CrunchyBridge) force `plugin.name: pgoutput` — no ability to install `decoderbufs`. Set it explicitly rather than relying on a default.

1.3 Create a replication user

```
CREATE ROLE <USER> WITH LOGIN PASSWORD '<password>' REPLICATION;
```

REPLICATION is the role attribute that actually lets the user open a replication connection — without it Postgres refuses the connector outright, no matter what table grants you hand out. Best practice (and the point of this whole step) is a scoped role, not the instance superuser.

Verify:

```
SELECT rolname, rolreplication, rolcanlogin FROM pg_roles WHERE rolname = '<USER>';
```

Both flags should read `t`.

1.4 Grant only what the connector needs: SELECT, not ownership

It's tempting to think `<USER>` needs to *own* the table because the next step (`CREATE PUBLICATION`) requires ownership. It doesn't — that ownership requirement applies to whoever **runs the `CREATE PUBLICATION` statement**, which is a one-time admin action you (or a migration script) execute as the table owner or superuser. It is not a requirement on the connector's runtime role.

Once the publication already exists, `<USER>` never needs ownership at all:

- **Streaming changes** flows off the WAL via the replication protocol (`pgoutput`) — Postgres doesn't re-check table-level privileges for that, only that the role has **REPLICATION** and is attached to a slot reading a publication it's part of.
- **The initial snapshot** (and any later ad hoc/incremental snapshot) is a plain `SELECT * FROM table` under the hood, so that's the only privilege `<USER>` actually needs.

So skip the shared-ownership dance entirely and just grant **SELECT**:

```
-- Run as postgres superuser or the table's real owner – NOT as <USER>
GRANT USAGE ON SCHEMA <DB-Name> TO <USER>;
GRANT SELECT ON <DB-Name>.<Table-Name>, <DB-Name>.<Table-Name> TO <USER>;
```

Verify:

```
SELECT has_schema_privilege('<USER>', '<DB-Name>', 'USAGE');
SELECT has_table_privilege('<USER>', '<DB-Name>.<Table-Name>', 'SELECT');
```

Expect `t` for both. `<USER>` is never the owner, so there's no `tableowner` check to run anymore.

Trade-off to know about: because `<USER>` never owns anything, it also can't run `ALTER PUBLICATION ... ADD TABLE` itself later. Adding a new table to CDC becomes a deliberate admin action (owner/superuser), not something the connector can do unattended. For most setups that's the more correct posture anyway — schema/CDC-scope changes shouldn't be self-service for a service account.

1.5 Create the publication

`pgoutput` streams changes for tables that belong to a *publication* — a Postgres-native logical-replication concept with no Mongo equivalent. It's best practice to create this yourself ahead of time rather than lean on Debezium's autcreate:

```
-- Run as postgres superuser or the table's real owner – this is the one
step
-- in the whole runbook that genuinely requires ownership, and <USER>
doesn't
-- need to be that owner for it to happen.
CREATE PUBLICATION dbz_publication FOR TABLE <DB-Name>.<Table-Name>, <DB-
Name>.<Table-Name>;
```

Match this to the connector's `publication.name` and `table.include.list` — if a table isn't in the publication, Debezium never sees its changes even if it's listed in `table.include.list`.

Verify:

```
SELECT pubname, puballtables FROM pg_publication;
SELECT * FROM pg_publication_tables;
```

`puballtables` should read `f` (scoped, not all-tables), and both target tables should show up in `pg_publication_tables`.

If you're following the SELECT-only model from 1.4 (publication manually created and owned by an admin, connector role never owns anything), set `publication.autocreate.mode: disabled` in the connector config — not `filtered`. `filtered` reconciles the publication's table list on every start via an

ownership-requiring `ALTER PUBLICATION`, which will fail against a SELECT-only role even though the publication already exists (see the parameter breakdown in section 2). `disabled` tells Debezium to use the publication exactly as you left it and never issue DDL against it.

If Debezium is left to autocreate instead (i.e. you skip this manual step and grant `CREATE`), set `publication.autocreate.mode: filtered` — the default (`all_tables`) tries `CREATE PUBLICATION ... FOR ALL TABLES`, which fails outright if the user lacks ownership on every table in the database (the default failure mode on CrunchyBridge specifically, so set `filtered` there as a matter of course). But note this is a different permission model from 1.4 — it requires the connector role to hold `CREATE ON DATABASE` and, for `filtered`'s ongoing reconciliation, to own the tables too.

1.6 Create the replication slot

The slot is what guarantees no data loss — Postgres retains every WAL segment the slot hasn't confirmed as processed, even across a Debezium outage. That guarantee cuts both ways: an orphaned slot (connector gone, slot still there) silently accumulates WAL and eats disk forever, so name it deliberately and know what it's for.

```
SELECT pg_create_logical_replication_slot('dbz_<Table-Name>_slot',  
'pgoutput');
```

Match `dbz_<Table-Name>_slot` to the connector's `slot.name`. Each connector needs its own uniquely-named slot — sharing one across connectors doesn't work.

Verify:

```
SELECT slot_name, plugin, slot_type, active FROM pg_replication_slots;
```

Expect the new row with `active = f` — it only flips to `t` once a connector actually attaches.

Manual creation (over letting Debezium auto-create on first connect) is the right call for: production generally, a controlled Postgres version upgrade where you need to preserve change data across it, failover recovery where the slot needs to exist on the new primary before writes resume, or a connector user that intentionally lacks `REPLICATION/CREATE`.

1.7 Allow the replication connection

Postgres authorizes replication-protocol connections separately from ordinary SQL connections, even for the same user. In `pg_hba.conf`, add an entry for the connector's host:

```
host      replication      <USER>      <connector-host-ip>/32      scram-sha-256
```

then reload (`SELECT pg_reload_conf();` — no restart needed for `pg_hba.conf` changes).

This line is not optional and nothing else substitutes for it. Postgres's `all` keyword in the `database` field explicitly does **not** match replication connections — that's been the documented behavior since 9.1, specifically to stop a generic `host all all all ...` line from silently granting replication access. So even a wide-open dev `pg_hba.conf` with `host all all all md5` will still reject Debezium with **no `pg_hba.conf` entry for replication connection** until you add the `replication` line above explicitly.

Verify — do this before ever touching Kafka Connect, since it isolates auth from everything else:

```
psql "host=<postgres-host> dbname=<DB-Name> replication=database user=
<USER>" -c "IDENTIFY_SYSTEM;"
```

If `pg_hba.conf` is wrong, this fails immediately with **no `pg_hba.conf` entry for replication connection**. If it's right, you get back a row with the system identifier and current WAL location — proof the auth path works independent of Debezium.

1.8 Replica identity — the important gotcha

By default (`REPLICA IDENTITY DEFAULT`), `UPDATE/DELETE` events only carry the **primary key** columns in **before** — not the full old row. If downstream needs the complete previous row state — the direct equivalent of Mongo's `capture.mode: change_streams_update_full` — set:

```
-- Run as owner/superuser — same as the publication, this is a DDL
statement
-- and doesn't need to be run by <USER>.
ALTER TABLE <DB-Name>.<Table-Name> REPLICA IDENTITY FULL;
```

Without this, don't expect **before** to help you diff old vs. new values; you'll only get the PK.

Verify:

```
SELECT relreplident FROM pg_class WHERE oid = '<DB-Name>.<Table-
Name>'::regclass;
```

f = FULL, **d** = DEFAULT (PK-only).

1.9 Final checklist before handing off

Run all of these together as a last pass — if everything here comes back clean, the Kafka Connect side (Section 2) will register without surprises:

```
SELECT rolname, rolreplication FROM pg_roles WHERE rolname = '<USER>';
SELECT has_table_privilege('<USER>', '<DB-Name>.<Table-Name>', 'SELECT');
SELECT pubname, puballtables FROM pg_publication;
SELECT slot_name, plugin, active FROM pg_replication_slots;
```

```
SELECT relreplident FROM pg_class WHERE oid = '<DB-Name>.<Table-Name>'::regclass;
```

2. Install the connector on Kafka Connect

Drop the Debezium PostgreSQL connector JAR into your Connect plugin path (e.g. `<kafka-plugins-path>/debezium-connector-postgres/`), restart the Connect workers.

Register via REST API:

```
curl -X POST http://connect-host:8083/connectors \
-H "Content-Type: application/json" \
-d @postgres-connector.json
```

`postgres-connector.json`:

```
{
  "name": "postgres-cdc-connector",
  "config": {
    "connector.class":
"io.debezium.connector.postgresql.PostgresConnector",
    "database.hostname": "<postgres-host>",
    "database.port": "5432",
    "database.user": "<USER>",
    "database.password": "<password>",
    "database.dbname": "<DB-Name>",
    "topic.prefix": "dbserver1",
    "table.include.list": "<DB-Name>.<Table-Name>,<DB-Name>.<Table-Name>",
    "plugin.name": "pgoutput",
    "slot.name": "dbz_<Table-Name>_slot",
    "publication.name": "dbz_publication",
    "publication.autocreate.mode": "disabled",
    "snapshot.mode": "initial",
    "tombstones.on.delete": "true",
    "key.converter": "org.apache.kafka.connect.json.JsonConverter",
    "value.converter": "org.apache.kafka.connect.json.JsonConverter",
    "key.converter.schemas.enable": "false",
    "value.converter.schemas.enable": "false"
  }
}
```

What each parameter controls:

- `database.hostname` / `port` / `dbname` — single primary connection, unlike Mongo's multi-host replica-set string. Postgres logical slots only live on a primary; you can't point the connector at a replica.

- `topic.prefix` — same role as in Mongo: prefix for generated topic names, but the naming pattern is `topicPrefix.schemaName.tableName` (schema-qualified), e.g. `dbserver1.<DB-Name>.<Table-Name>` — one topic per table.
- `table.include.list` — whitelist, same idea as Mongo's `collection.include.list`. Fully-qualified `schema.table`.
- `plugin.name` — `pgoutput` or `decoderbufs`. Must match how the server/plugin was set up.
- `slot.name` — the replication slot this connector owns. One connector = one slot, and the slot persists on the server between connector restarts, holding WAL until Debezium confirms it's processed. **A slot with no connector reading from it is a live WAL-bloat risk** — Postgres won't reclaim WAL segments the slot hasn't confirmed past, even if the connector is gone for good.
- `publication.name` — must match (or be creatable as) the publication from step 1.5.
- `publication.autocreate.mode` — three real options, and the choice matters more than it looks:
 - `all_tables` (default) — tries `CREATE PUBLICATION ... FOR ALL TABLES` if none exists. Fails outright if the connector user lacks ownership on every table in the database.
 - `filtered` — scopes an auto-created publication to `table.include.list`. **Since Debezium 1.9.7, this mode doesn't only create — it also reconciles an existing publication's table membership on every connector start**, via `ALTER PUBLICATION ... SET TABLE`. That's an ownership-requiring statement, so `filtered` will fail with `must be owner of publication` even against a publication you pre-created, if the connector user only has `SELECT`.
 - `disabled` — Debezium never touches the publication's DDL at all; it just uses whatever already exists under `publication.name`. **This is the right setting for the SELECT-only model in 1.4/1.5** — the publication is created and owned by an admin once, and the connector user is never expected to alter it.
- `snapshot.mode` — same menu of values as Mongo conceptually (`initial`, `no_data`, `never/initial_only` etc.), plus a few Postgres-only modes (`always`, `when_needed`, `configuration_based`) for things like recovering after a failover promotes a new primary.
- `tombstones.on.delete` — same as Mongo: null-value marker after deletes for Kafka log compaction, filtered out in NiFi.

Check it came up:

```
curl http://connect-host:8083/connectors/postgres-cdc-connector/status
```

3. Event shape

```
{
  "before": null,
  "after": {
    "id": 1,
    "status": "shipped",
    "amount": 250
  },
  "op": "u",
```

```
"ts_ms": 1752480000123
}
```

Key difference from Mongo: **after** (and **before**) are already structured JSON objects, not JSON-encoded strings. No double-parsing needed in NiFi — one **EvaluateJsonPath** pass gets you there.

op values (Postgres has two more than Mongo):

- **r** — snapshot read (initial load, treat as insert)
- **c** — insert
- **u** — update
- **d** — delete
- **t** — truncate (whole table wiped; no message key, needs its own handling — see below)
- **m** — generic logical decoding message (only via **pgoutput** on Postgres 14+, rare unless something explicitly calls **pg_logical_emit_message**)

before on **u/d** events only contains what **REPLICA IDENTITY** exposes — PK columns only by default, full row if you set **REPLICA IDENTITY FULL** (see 1.8).

Also note: a primary-key **update** doesn't come through as a single **u** event — Debezium splits it into a **d** (old key) + **c** (new key), each carrying a message header (**__debezium.newkey** / **__debezium.oldkey**). If your NiFi flow assumes PK is immutable per row, this is the edge case that breaks that assumption.

Notes

- **WAL disk bloat is the #1 operational headache with Postgres CDC**, more so than with Mongo's oplog. A replication slot holds onto WAL segments until Debezium confirms it has processed past them. If the connector is down, misconfigured, or just slow, WAL piles up on the server disk — potentially filling it. Check **pg_replication_slots**: if **confirmed_flush_lsn** isn't advancing while **restart_lsn** lags behind, the server isn't reclaiming space and something downstream is stuck.
- Drop the replication slot manually (**SELECT pg_drop_replication_slot('dbz_<Table-Name>_slot')**) if you're permanently decommissioning a connector — an orphaned slot doesn't clean itself up and will quietly eat disk indefinitely.
- If Connect restarts, it resumes from the last committed LSN (the offset), same no-replay guarantee as Mongo's committed Kafka offset — as long as the WAL segment covering it hasn't been discarded, which ties back to the point above.
- Postgres has a fixed schema per table (unlike Mongo), so schema drift shows up differently: logical decoding doesn't propagate DDL changes to the event stream, so adding/dropping/renaming columns won't automatically flow through — you'll need to update **table.include.list**/downstream schema handling or trigger an ad hoc (incremental/blocking) snapshot to pick up the new shape.
- Set backpressure thresholds on the queue after **ConsumeKafka**, same reasoning as Mongo — a slow sink shouldn't be allowed to pile up unbounded memory in NiFi.
- Reference: [Debezium PostgreSQL connector docs](#)